

Python Pipeline for Six-Class Alzheimer MRI Classification

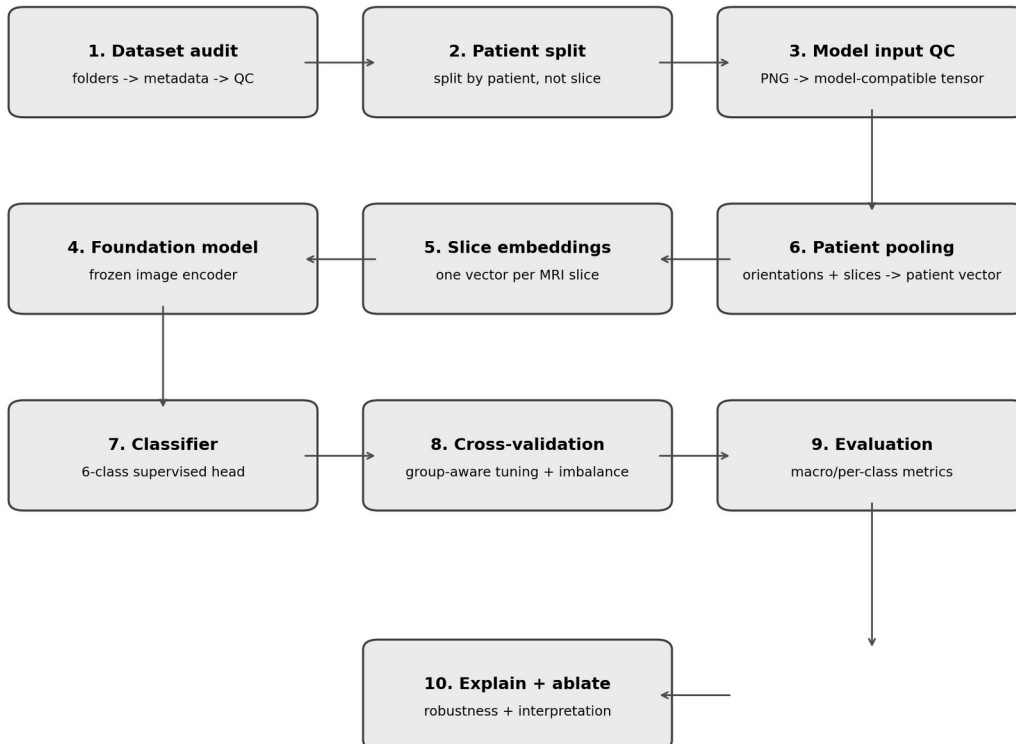
Foundation-model centered workflow for 2D preprocessed PNG slices

Study classes

AD | CN | EMCI | LMCI | MCI | SMC

Main methodological change from the reference paper:

No beta-amyloid stratification as a target or additional class. No MATLAB.
The main experiment uses foundation-model image embeddings in Python.



1. Purpose and study design

Research question. Can pretrained foundation-model representations of structural MRI slices be used effectively for a six-class classification problem: AD, CN, EMCI, LMCI, MCI and SMC? The core analysis should test whether the learned image representation contains information useful for discriminating these six groups, while keeping the final classifier and validation procedure transparent.

Reference-paper relationship: The 2026 paper uses T1-weighted sMRI, multi-plane slicing, DWT2, histogram and GLCM descriptors, feature compression, normalization, feature selection and supervised ML. Your project keeps the six clinical groups and the multi-plane idea, but removes beta-amyloid stratification, replaces MATLAB with Python, and makes foundation-model embeddings the main representation.

Primary analysis unit: the patient. Individual PNG slices are observations used to construct a patient-level representation; they should not be treated as independent patients during splitting or final evaluation.

- Dataset classes: AD, CN, EMCI, LMCI, MCI, SMC.
- Image format currently available: 2D PNG slices, already preprocessed.
- Approximate current file count: about 73,000 image files. The exact patient count should be computed from metadata, not inferred from the image count.
- No beta-amyloid variable is required for the main target. If beta-amyloid metadata exist, keep it outside the target label unless you later design a separate secondary analysis.
- The final reported model should be selected and tuned without using the held-out test patients.

2. Dataset inventory and metadata layer

Before feature extraction, create one metadata table with one row per PNG slice. This table is the backbone of the entire project: it connects files, patients, orientations, slice numbers, labels, splits and later embeddings.

Field	Meaning	Example	Why it matters
image_path	Full path to PNG	.../SMC/1306/1306_sagittal_slice_196.png	Load image
diagnosis	Class from first folder	SMC	Target y
patient_id	Patient identifier	1306	Group-aware split
orientation	axial / coronal / sagittal	sagittal	Multi-plane analysis
slice_number	Slice index from filename	196	Ordering and QC
filename	Original filename	1306_sagittal_slice_196.png	Traceability
split	train / validation / test	train	Leakage control
embedding_path	Saved vector location	embeddings/...	Model output linkage

Recommended metadata checks

- Verify that every image maps to exactly one of the six allowed class labels.
- Verify that each filename can be parsed into patient ID, orientation and slice number.
- Check duplicated paths, duplicated filenames, missing patient IDs and unexpected extensions.
- Count images per patient and per orientation. Very large differences can matter when you aggregate slices.
- Check whether the same patient ID appears in more than one diagnostic folder. If it does, investigate before modeling.

3. Image quality control

Because your data are already preprocessed PNGs, do not automatically repeat the paper's raw-volume preprocessing. Instead, verify that the existing output is consistent and suitable for the chosen foundation model.

QC item	What to inspect	Python tools	Decision
Image dimensions	All slices have expected dimensions	Pillow, NumPy	Flag unexpected sizes
Image mode	Grayscale vs RGB, bit depth	Pillow	Standardize only when required

QC item	What to inspect	Python tools	Decision
Pixel distribution	min, max, mean, std, histogram	NumPy, Matplotlib	Find blank/saturated/outlier slices
Visual orientation	axial/coronal/sagittal look plausible	Pillow, Matplotlib	Sample each class + orientation
Blank/near-blank slices	Very low information content	NumPy	Decide exclusion rule before training
Corrupted files	Cannot be opened or decoded	Pillow	Remove/log and report count

Important: Do not decide to exclude slices merely because they look less informative without defining a reproducible rule. The rule should be decided before model evaluation and applied consistently.

4. Patient-level data splitting - the critical leakage barrier

This is the most important statistical design rule in the project. A patient contributes many slices, often from three orientations. If slices from the same patient enter both training and test sets, the model can exploit patient-specific characteristics and the measured performance can be substantially misleading.

Recommended structure

- Construct a patient table: one row per patient with patient_id and diagnosis.
- Create train/validation/test assignments at patient level.
- Map the patient assignment back to all of that patient's slice rows.
- Never move an individual slice between splits.
- If using cross-validation instead of a fixed test set, use a group-aware splitter with patient_id as the group variable.

Stage	Recommended Python concept	Rule
Fixed hold-out	Stratified split on patient table	Stratify by diagnosis; group already enforced by patient table
Cross-validation	StratifiedGroupKFold	Groups = patient_id; class labels = diagnosis
Final test	Untouched patient set	Used only once for final evaluation
Hyperparameter tuning	Grid/RandomizedSearchCV inside training data	Never tune on the final test set

Embedding timing: For the cleanest experimental design, freeze the patient split before any task-specific adaptation, fine-tuning, feature selection, scaling, PCA or classifier tuning. A completely frozen pretrained encoder can be applied to all images without using labels, but patient-level separation should still be preserved throughout the experiment.

5. Model-input preparation for 2D PNG images

The existing preprocessing pipeline is assumed to have produced anatomically aligned and brain-focused slices. Your next task is not to recreate the 3D preprocessing from the paper; it is to make each PNG conform to the selected foundation model's input contract.

Operation	Purpose	Python library	Key functions / concepts
Read image	Load PNG reliably	Pillow	Image.open
Convert channels	Map grayscale input to model-required channels	Pillow, torchvision	convert; model-specific channel handling
Resize/crop	Match model input size	torchvision	Resize, CenterCrop or model processor
Tensor conversion	Convert image to numeric tensor	torchvision	ToTensor / processor
Normalization	Match model training distribution	torchvision / model processor	Use exactly the preprocessing specified by the model
Batching	Efficient inference	PyTorch	DataLoader, batch dimension

Do not guess normalization parameters: The correct mean/std, channel convention, image size and interpolation depend on the chosen foundation model. Read the model card or official processor definition and reproduce it exactly. Do not automatically apply ImageNet normalization to every medical model.

6. Foundation-model layer

The foundation model is used as an image representation engine. For the first experiment, keep the encoder frozen and extract embeddings. This separates the question "are the pretrained representations useful?" from the harder question "can the entire model be fine-tuned on this relatively small patient cohort?"

Model option	Type	How to use it	Strength for this study	Caution
MedImageInsight v2	Medical imaging embedding model	Extract image embeddings; train your six-class classifier on them	Explicitly designed for medical imaging and supports MRI	Verify current access/deployment route and exact preprocessing; document model version
DINOv3 ViT-B/16	General vision foundation model	Frozen visual embeddings + downstream classifier	Strong modern self-supervised vision baseline with simple downstream adaptation	Pretrained on web imagery, not MRI-specific
BiomedCLIP	Biomedical image-text foundation model	Image embeddings or optional image-text experiment	Biomedical pretraining and multimodal representation	Text-image pretraining does not guarantee optimal MRI staging features

Practical starting point: Use one medical foundation model as the primary experiment and one general-purpose foundation model as a comparator. Keep the classifier, split strategy and evaluation protocol identical so that the representation itself is the main experimental difference.

Foundation-model output to retain for every slice

- patient_id
- diagnosis
- orientation
- slice_number
- image_path
- embedding vector
- model name and exact version/checkpoint
- preprocessing/version identifier

7. Slice-level embedding extraction

Run the frozen foundation model in inference mode over the PNG dataset. The output is a fixed-length vector for each image. The vector is not a hand-designed histogram or texture feature; it is a learned representation produced by the pretrained encoder.

Stage	Input	Output	Why
Load	PNG path	PIL image	Stable file access
Transform	PIL image	model-ready tensor	Match model contract
Encode	tensor batch	embedding matrix [batch x D]	Compact learned representation
Save	embedding + metadata	persistent dataset	Avoid recomputing 73k embeddings
Validate	sample embeddings	shape/dtype/checksum metadata	Catch model/configuration mistakes early

Python components

- PyTorch: tensor operations, no_grad/inference mode, device management and batching.
- torchvision or the model-specific processor: deterministic image preprocessing.
- Pillow: image loading.
- NumPy: save/load embedding matrices and inspect shapes.
- Pandas: keep the metadata table synchronized with embedding rows.
- Optional: tqdm for progress monitoring over a large image collection.

8. Convert thousands of slices into one patient representation

This is the second key design decision after the patient split. A six-class diagnosis is attached to the patient, not to an independently sampled slice. Therefore, the foundation-model slice embeddings must be aggregated into a patient-level representation before the final classifier is evaluated.

Recommended first patient representation

- Group embeddings by patient_id and orientation.
- Within each orientation, compute a robust summary across slices, starting with mean embedding.
- Optionally retain a second summary such as standard deviation or a high/low percentile of each embedding dimension to describe within-patient heterogeneity.
- Concatenate the axial, coronal and sagittal summaries into a single patient vector.
- Keep the orientation-specific representations so you can perform orientation ablations later.

Aggregation option	Level	Use	Complexity
Mean pooling	patient x orientation	Primary baseline	Low
Mean + standard deviation	patient x orientation	Captures intra-patient variation	Low-moderate
Top-k slice pooling	patient x orientation	Focuses on most informative slices	Moderate; needs a selection rule
Attention-based MIL	patient	Learns which slices matter	Higher; requires PyTorch training
3D volumetric model	patient	Direct 3D representation	Not appropriate for your current PNG-only setup without original 3D volumes

Primary recommendation: Begin with mean pooling because it is easy to understand, deterministic and hard to overfit. Only move to attention-based multiple-instance learning after you have a clean frozen-embedding baseline.

9. Six-class classifier on patient embeddings

After aggregation, the dataset becomes one row per patient and one target label per row. This is where conventional supervised machine learning can be used. The foundation model provides the representation; the classifier maps that representation to the six diagnostic groups.

Classifier	Python implementation	Role	Priority
Multinomial logistic regression	scikit-learn LogisticRegression	Transparent linear probe on embeddings	First model
Linear SVM	scikit-learn LinearSVC / SVC	Strong linear baseline	Second model
Random forest	scikit-learn RandomForestClassifier	Nonlinear tree ensemble	Comparator
Gradient boosting	scikit-learn HistGradientBoostingClassifier or XGBoost/LightGBM if added	Nonlinear alternative	Optional
Small MLP	PyTorch nn.Module	Learned nonlinear head	Later, after linear probes

Why start with a linear probe? A foundation model is already expected to produce a semantically useful representation. A linear classifier gives a controlled test of that representation and avoids immediately introducing another large trainable model.

10. Scaling, dimensionality reduction and feature selection

Embedding vectors can contain hundreds or thousands of dimensions. These steps are optional, but the pipeline should define them explicitly because applying them to the full dataset before splitting is a source of information leakage.

Operation	Python function/concept	Fit on	Purpose
Standardization	sklearn.preprocessing.StandardScaler	Training fold only	Equalize feature scale for linear models
PCA	sklearn.decomposition.PCA	Training fold only	Compress embedding dimensions / reduce noise
Univariate selection	SelectKBest + f_classif or mutual_info_classif	Training fold only	Reduce dimensions using labels
Regularization	LogisticRegression C / penalty	Training fold only	Control overfitting

Operation	Python function/concept	Fit on	Purpose
Tree-based importance	Random forest feature importance	Training fold only	Optional nonlinear feature screening

Leakage rule: Scaler, PCA, feature selector and resampling method are all learned from training patients only. In cross-validation they belong inside the fold pipeline, not as a preprocessing step run once on the entire dataset.

11. Class imbalance handling

The reference paper used resampling strategies for imbalanced binary tasks. Your main problem is six-class classification, so the first imbalance control should be class-aware training rather than automatically applying aggressive resampling to images.

Method	Python tool	Where	Comment
Class weights	scikit-learn <code>class_weight / compute_class_weight</code>	Training only	Simple first choice for logistic regression / SVM / some classifiers
Random under-sampling	imbalanced-learn <code>RandomUnderSampler</code>	Training only	Useful when a majority class dominates, but discards data
SMOTE	imbalanced-learn SMOTE	Training patient embeddings only	Generate synthetic vectors; never synthesize test data or leak across folds
No resampling + calibrated model	scikit-learn classifiers	Training only	Useful comparator; report class-specific metrics

SMOTE rule: Do not apply SMOTE to individual MRI slices and then pool them as if they were real patients. If SMOTE is tested, apply it after patient-level embedding aggregation and only inside the training fold.

12. Validation and experimental protocol

The validation framework should answer two questions: how well does the representation discriminate the six classes, and how stable is that performance across patient subsets?

Layer	Procedure	Data allowed	Purpose
Development split	patient-level train/test or train/validation/test	Patient labels only	Protect final evaluation
Model selection	StratifiedGroupKFold or nested CV	Training patients	Compare models/hyperparameters
Preprocessing fit	Scaler/PCA/selector/resampler inside fold	Training patients only	Prevent leakage
Final training	Refit selected pipeline on all development patients	Development set	Prepare final model
Final evaluation	One pass on held-out test patients	Test patients only	Estimate generalization

Recommended Python tools

- `sklearn.model_selection.StratifiedGroupKFold` for patient-aware cross-validation.
- `GridSearchCV` or `RandomizedSearchCV` for controlled hyperparameter tuning.
- `sklearn.pipeline.Pipeline` to keep transformation order explicit and leakage-safe.
- `imbalanced-learn Pipeline` when resampling is part of a fold-level workflow.

13. Evaluation measures to report

Because there are six classes and likely unequal class sizes, report both overall and class-balanced measures. Do not rely on accuracy alone.

Metric / output	Python function	What it tells you
Accuracy	<code>accuracy_score</code>	Overall fraction correct
Balanced accuracy	<code>balanced_accuracy_score</code>	Average recall across classes

Metric / output	Python function	What it tells you
Precision / recall / F1 per class	classification_report	Which stages are easier or harder to classify
Macro F1	f1_score(average="macro")	Equal importance to each class
Confusion matrix	confusion_matrix	Where six-class errors occur
One-vs-rest ROC-AUC	roc_auc_score(multi_class=...)	Ranking/discrimination across classes
Prediction probabilities	predict_proba where supported	Required for ROC-AUC and calibration checks

Interpretation rule: For a six-class disease continuum, inspect the confusion matrix and per-class recall together. Adjacent categories such as EMCI/MCI/LMCI are scientifically different from broad AD-vs-CN separation, so the aggregate score should never be the only reported quantity.

14. Interpretation and explainability

A foundation model gives you a learned representation, but a clinically meaningful study still needs evidence about what the model is using and whether it is relying on unwanted artifacts.

Analysis	Python tools	Use	Level
Embedding visualization	PCA; UMAP (umap-learn)	Visualize class structure in representation space	Exploratory
Nearest-neighbor inspection	NumPy / scikit-learn NearestNeighbors	See which slices/patients are represented as similar	Exploratory + QC
Permutation importance	sklearn.inspection.permutation_importance	Assess contribution of aggregated embedding dimensions	Model-level
Attention/MIL maps	PyTorch + model-specific attention extraction	Identify influential slices if using attention MIL	Advanced
Gradient-based explanation	Captum; model-specific CAM/attention methods	Local explanation for a trainable vision model	Advanced

Artifact check: Compare model attention or explanation patterns with the image itself. A model can learn scanner borders, text, slice position, acquisition artifacts or other shortcuts. Explanations are diagnostic tools, not proof that the model has learned disease biology.

15. Experiments that make the study scientifically interpretable

Do not change several methodological factors at once. Build the study as a sequence of controlled comparisons.

Experiment	Keep constant	Change	Question
A. Primary foundation model	split, aggregation, classifier, metrics	Medical foundation model	Can a medical pretrained representation solve the six-class task?
B. General foundation baseline	same as A	DINOv3 or equivalent general vision FM	Does medical pretraining add value on these images?
C. Orientation ablation	same model/classifier	axial only / coronal only / sagittal only / all three	Which planes carry useful information?
D. Pooling ablation	same model/split	mean vs mean+std vs attention MIL	How important is slice aggregation?
E. Linear vs nonlinear head	same embeddings	logistic regression vs SVM vs tree/MLP	Is the embedding linearly separable?
F. Fine-tuning	same split and target	frozen vs adapter/fine-tuned encoder	Does task-specific adaptation help beyond frozen features?
G. Classical comparator	same patients and evaluation	DWT + histogram + GLCM features	How does learned representation compare with hand-crafted radiomics?

16. Optional second phase: fine-tuning / adapter training

Only after the frozen-embedding experiment is stable should you train part of the foundation model. With a patient-level dataset that is much smaller than the number of slices, full fine-tuning of a large foundation model can overfit.

- Freeze the backbone initially and train only the classifier head.

- Next, consider parameter-efficient adaptation if the chosen model supports it.
- Use a small learning rate and strong regularization when unfreezing layers.
- Perform all augmentation only on training images/patients. Never augment the validation/test set for evaluation.
- Use patient-aware folds throughout fine-tuning; do not create separate folds at the slice level.
- Save the exact model checkpoint, preprocessing configuration and random seed for every experiment.

17. Optional Python reproduction of the paper's hand-crafted feature branch

This branch is not the main foundation-model analysis. It is useful as a scientifically meaningful comparator because it translates the paper's MATLAB feature pipeline into Python.

Paper component	Python library	Key functions / concepts	Notes
2D DWT	PyWavelets (pywt)	wavedec2 / dwt2, wavelet=bior1.1, level=2	Use the exact decomposition convention you document
Histogram statistics	NumPy + SciPy	mean, median, percentile, std, var, skew, kurtosis, min, max, entropy etc.	Compute per slice/subband
GLCM	scikit-image	graycomatrix, graycoprops; custom calculations for measures not directly provided	Use paper offsets and distance if reproducing it
Slice-to-patient compression	NumPy/Pandas	groupby + descriptive summaries	Mean, median, std, variance, Q05, Q95, skew, kurtosis
Feature scaling	scikit-learn	StandardScaler	Fit on training fold only
Feature selection	scikit-learn	SelectKBest, f_classif, mutual_info_classif, chi2; model-based selectors	Fit on training fold only
Classification	scikit-learn	LogisticRegression, SVC, RandomForestClassifier, HistGradientBoostingClassifier	Use same patient-level validation as foundation model

Paper settings worth documenting when reproducing the branch

- Two-level DWT with the bior1.1 wavelet.
- Multiple anatomical planes: axial, coronal and sagittal.
- GLCM distance $d = 1$ with four offsets: $[1,0]$, $[-1,1]$, $[-1,0]$, $[-1,-1]$, followed by averaging across offsets.
- The paper computes first-order histogram statistics and GLCM texture descriptors for wavelet-derived images.
- The paper then compresses slice-level measurements using mean, median, standard deviation, variance, 5th and 95th percentiles, skewness and kurtosis.
- Normalization and feature selection are restricted to training data to avoid leakage.

Reproduction caution: The paper text describes a wavelet collection containing 8 explicitly listed images/components, but later feature-count equations use 9 wavelet sub-bands. Resolve this discrepancy from the authors' implementation or supplementary material before claiming an exact reproduction.

18. Python package map - what each package is for

Package	Role in project	Functions / classes to learn	Essential?
os / pathlib	File discovery and paths	os.walk, os.path.join, pathlib.Path.rglob	Yes
pandas	Metadata/data table	DataFrame, read_csv, groupby, merge, value_counts, to_parquet	Yes
Pillow	PNG loading/QC	Image.open, convert, size, mode	Yes
NumPy	Arrays, numerical summaries, embeddings	array, mean, std, percentile, save/load, concatenate	Yes
Matplotlib	Image and distribution inspection	imshow, hist, figure, show	Yes
SciPy	Statistics for classical comparator	scipy.stats.skew, kurtosis	Optional for FM branch
scikit-image	GLCM and image analysis	graycomatrix, graycoprops	Optional for comparator
PyWavelets	DWT features	wavedec2, dwt2	Optional for comparator
PyTorch	Foundation-model inference/fine-tuning	model, tensor, DataLoader, inference_mode, device	Yes for FM
torchvision	2D transforms and vision utilities	Resize, CenterCrop, ToTensor, transforms	Usually yes for 2D vision

Package	Role in project	Functions / classes to learn	Essential?
scikit-learn	Classifiers, scaling, CV, metrics	Pipeline, StandardScaler, PCA, LogisticRegression, SVC, RandomForestClassifier, StratifiedGroupKFold, metrics	Yes
imbalanced-learn	Resampling	SMOTE, RandomUnderSampler, pipeline integration	Optional
tqdm	Progress display	tqdm	Optional
umap-learn	Embedding visualization	UMAP	Optional
captum	Neural-network explanations	Attribution methods	Optional

19. Recommended project structure

Keep the project modular so that data preparation, embedding extraction and modeling can be run independently. This is especially important because extracting 73k embeddings is computationally expensive and should not have to be repeated every time you change a classifier.

data/ Original or linked dataset locations; keep source data unchanged.

metadata/ Master slice table and patient table.

qc/ Quality-control logs and sample visualizations.

embeddings/ Saved foundation-model embeddings plus model/version metadata.

models/ Trained classifier or fine-tuned checkpoints.

experiments/ Configuration files and experiment definitions.

figures/ QC figures, embedding plots, confusion matrices and explanations.

reports/ Tables and final written outputs.

src/ Python modules for metadata, preprocessing, embeddings, aggregation, modeling and evaluation.

20. The exact order to follow

Step	Action	Output
01	Finish metadata table	image_path, diagnosis, patient_id, orientation, slice_number
02	Run dataset QC	counts, dimensions, image modes, blank/corrupt files
03	Create patient-level split	assign train/validation/test before model development
04	Choose primary foundation model	document name, version, checkpoint, license and input preprocessing
05	Run one-image model test	confirm that one PNG enters the model and one embedding comes out
06	Extract all slice embeddings	batch inference, save embeddings, retain metadata linkage
07	Aggregate at patient level	orientation-wise mean pooling; concatenate planes
08	Train linear classifier	logistic regression as primary probe; keep class handling explicit
09	Run group-aware CV	tune only on training patients; all learned transforms inside folds
10	Evaluate final held-out patients	accuracy, balanced accuracy, macro F1, per-class metrics, confusion matrix, ROC-AUC
11	Run orientation and pooling ablations	identify whether performance depends on one plane or pooling rule
12	Add second foundation model	keep all downstream steps identical
13	Optional classical comparator	DWT + histogram + GLCM implemented in Python

Step	Action	Output
14	Optional fine-tuning	only after frozen-embedding baseline is stable
15	Write up limitations	dataset size, class imbalance, possible pretraining overlap, 2D vs 3D representation, scanner/site effects

21. Decisions you should make explicitly in the methodology section

- What constitutes one patient and how patient IDs are determined.
- How the train/validation/test split is generated and the random seed used.
- Which foundation model(s) are tested and the exact checkpoint/version.
- How grayscale PNGs are converted to the model's expected channel format.
- Which resizing and intensity normalization the model processor uses.
- Whether slices are pooled across each orientation separately before combining orientations.
- Whether the classifier receives mean-only or mean-plus-variability patient embeddings.
- Whether class weighting, under-sampling or SMOTE is used, and at what stage.
- Which hyperparameters are tuned and inside which cross-validation procedure.
- Which metrics define model comparison and which are reported for every class.
- Whether any raw MRI or ADNI metadata were available to the foundation model during pretraining, and how this uncertainty is handled in the limitations.
- Which ablation experiments are pre-specified rather than selected after seeing the final test performance.

References and official resources

- Menezes, J., Barbosa, M. I., & Rodrigues, P. M. (2026). beta-amyloid-stratified six-stage Alzheimer's disease discrimination via multiband MRI histogram and GLCM features. Biomedical Signal Processing and Control, 127, 111143. DOI: 10.1016/j.bspc.2026.111143.
- Codella, N. C. F. et al. (2024). MedImageInsight: An Open-Source Embedding Model for General Domain Medical Imaging. arXiv:2410.06542.
- Microsoft Foundry. MedImageInsight model catalog, version 2 (updated 2026-09-03). Medical-image embedding model supporting classification and MRI inputs.
- Meta AI / facebookresearch. DINOv3 model card and repository. DINOv3 is a family of self-supervised vision foundation models and supports frozen-feature downstream classification.
- Zhang, S. et al. (2024). BiomedCLIP: a multimodal biomedical foundation model pretrained from fifteen million scientific image-text pairs. Microsoft Research / arXiv.
- Python package documentation to consult during implementation: Pillow, NumPy, pandas, Matplotlib, SciPy, scikit-image, PyWavelets, PyTorch, torchvision, scikit-learn and imbalanced-learn.

Note on scope: This document is intentionally a pipeline and implementation map rather than a code tutorial or a report of model results. It uses the reference paper as the methodological scaffold and adapts it to the stated six-class, 2D PNG, Python, foundation-model study design.

Official / article links:

<https://doi.org/10.1016/j.bspc.2026.111143>

<https://huggingface.co/papers/2410.06542>

<https://ai.azure.com/catalog/models/MedImageInsight>

<https://github.com/facebookresearch/dinov3>

<https://www.microsoft.com/en-us/research/publication/biomedclip-a-multimodal-biomedical-foundation-model-pretrained-from-fifteen-million-scientific-image-text-pairs/>